Sensors & Sensing Lab 6 (LiDAR, ICP, RANSAC, ROS 2)

This report documents the implementation and validation of:

1. Multi-snapshot 3D map building with ICP.
2. ICP comparison: custom NumPy ICP vs Open3D ICP.
3. Real-time floor and wall detection using iterative RANSAC.
4. RViz2 visualization of segmentation and map publishing.

1) Platform and Setup

- Sensor: Livox MID-70.
- ROS 2: Humble.
- Main workspace: LAB06_Lidar/ros2_ws.
- Offline processing and experiments: Lab_6_LiDAR_ROS2.ipynb.

Build commands used:

```
cd LAB06_Lidar/ros2_ws
colcon build --packages-select lidar_lab
source install/setup.bash
```

2) Task 2 - Point Cloud Registration and Map Stitching

2.1 Snapshot capture

Multiple LiDAR snapshots were captured from different positions with overlap between views.

Command used per snapshot:

```
ros2 bag record -o snapshot_<N> /livox/lidar
```

Alternative timed capture:

```
timeout 10 ros2 bag record -o snapshot_<N> /livox/lidar
```

2.2 Rosbag to dense point cloud (.ply)

The notebook converts PointCloud2 messages to NumPy arrays, accumulates each snapshot, and exports to .ply.

Main steps:

1. Read /livox/lidar messages from rosbag.
2. Decode PointCloud2 x,y,z fields.
3. Filter invalid points.
4. Save accumulated cloud as .ply.

2.3 Preprocessing

Before registration, each snapshot is preprocessed with:

1. Voxel downsampling.
2. Statistical outlier removal.
3. Normal estimation.

Goal: reduce noise and runtime while keeping geometric structure for robust ICP.

2.4 Custom ICP (NumPy)

Implemented point-to-point ICP includes:

1. KDTree nearest-neighbor matching.
2. SVD-based rigid transform estimation.
3. Iterative refinement with convergence tolerance.

2.5 Open3D ICP comparison

Open3D baselines were evaluated with:

1. Point-to-Point ICP.
2. Point-to-Plane ICP.

Comparison criteria:

1. Iteration behavior.
2. Inlier RMSE.
3. Fitness.
4. Runtime.

2.6 Multi-snapshot stitching

Clouds are aligned pairwise and merged into a single room map. The workflow includes quality checks for candidate transforms and optional rejection of poor alignments.

Output files:

1. room_map_3d.ply
2. room_map_3d_colored.ply
3. room_map_3d_tuned.ply
4. room_map_3d_colored_tuned.ply

2.7 Publish map to RViz2

A ROS 2 node publishes the stitched map as PointCloud2 on /map_3d.

Run:

```
ros2 run lidar_lab map_publisher
```

or

```
ros2 launch lidar_lab map_publisher.launch.py
```

3) Task 3 - Real-Time Wall and Floor Detection (RANSAC)

Package: lidar_lab

Node: plane_detector_node.py

Launch:

```
ros2 launch lidar_lab plane_detector.launch.py
```

3.1 Node behavior

Per incoming frame:

1. Convert PointCloud2 to NumPy xyz.
2. Apply voxel downsampling.
3. Run iterative RANSAC plane extraction.
4. Classify planes using normal direction:
5. Publish floor points, wall points, object points, and markers.

Plane labeling logic:

1. Floor: |n_z| above floor threshold.
2. Wall: |n_z| below wall threshold.
3. Other: otherwise.

3.2 Published topics

1. /planes/floor (PointCloud2)
2. /planes/walls (PointCloud2)
3. /planes/objects (PointCloud2)
4. /planes/markers (MarkerArray)
5. /map_3d (PointCloud2)

3.3 RViz2 configuration

Add displays:

1. PointCloud2: /planes/floor
2. PointCloud2: /planes/walls
3. PointCloud2: /planes/objects
4. MarkerArray: /planes/markers
5. PointCloud2: /map_3d

4) Required Deliverables

4.1 Combined 3D map in RViz2

[SCREENSHOT PLACEHOLDER - RViz2 combined color-coded map]

4.2 ICP comparison table

Fill the values from notebook output:

Method	Iterations to converge	RMSE (m)	Fitness	Execution time (s)
Custom ICP (NumPy, P2P)	[FILL]	[FILL]	N/A or [FILL]	[FILL]
Open3D ICP (Point-to-Point)	[FILL]	[FILL]	[FILL]	[FILL]
Open3D ICP (Point-to-Plane)	[FILL]	[FILL]	[FILL]	[FILL]

4.3 Convergence plots

[SCREENSHOT PLACEHOLDER - RMSE vs iteration plot]

4.4 Real-time RANSAC segmentation in RViz2

[SCREENSHOT PLACEHOLDER - RViz2 floor/wall/object point clouds]

[SCREENSHOT PLACEHOLDER - RViz2 colored MarkerArray labels]

5) Answers to Lab Questions

Q1) ICP convergence

Custom ICP convergence and Open3D performance were compared using RMSE, fitness, and runtime.

Observation template:

1. Custom ICP converged in [FILL] iterations.
2. Open3D Point-to-Point RMSE was [FILL] and runtime [FILL] s.
3. Open3D Point-to-Plane RMSE was [FILL] and runtime [FILL] s.
4. Point-to-Plane converged faster because normals constrain motion along local surfaces, reducing search ambiguity.

Q2) ICP sensitivity

Sensitivity experiments summary:

1. Lower overlap reduced alignment quality and increased failure risk.
2. Skipping preprocessing increased noise, slowed runtime, and degraded stability.
3. Poor initial guess (for example, 45 degrees yaw) worsened RMSE/fitness and may converge to local minima.

Experiment evidence placeholders:

1. [SCREENSHOT PLACEHOLDER - poor initial guess metrics]
2. [SCREENSHOT PLACEHOLDER - reduced overlap or no preprocessing metrics]

Q4) RANSAC reliability (10 runs)

Expected behavior:

1. Results are similar but not always identical due to random sampling.
2. The most influential repeatability parameters are num_iterations and distance_threshold.
3. More iterations and a well-tuned threshold improve consistency but increase compute time.

Run log placeholder:

1. [SCREENSHOT PLACEHOLDER - 10-run reliability summary]

Q5) Plane classification quality

Qualitative assessment template:

1. Successfully detected walls: [FILL].
2. Missed or unstable walls: [FILL].
3. Main causes: occlusion, long distance, grazing angle, sparse returns, reflective/absorptive materials.

Evidence placeholder:

1. [SCREENSHOT PLACEHOLDER - detected vs missed walls in RViz2]

Q6) Real-time performance bottleneck

Pipeline timing (as printed by node logs) includes downsample, ransac, publish, total.

Conclusion template:

1. Measured bottleneck: [FILL].
2. Why: [FILL].
3. Optimization options: reduce points via voxel size, tune max_planes, tune num_iterations, vectorize and reduce per-frame allocations.

Timing placeholder:

1. [SCREENSHOT PLACEHOLDER - per-frame timing logs]

6) Reproducibility Checklist

1. Build package in LAB06 workspace.
2. Run map publisher and verify /map_3d in RViz2.
3. Run plane detector and verify /planes/floor, /planes/walls, /planes/objects, /planes/markers.
4. Run notebook cells for ICP comparison and export figures.
5. Replace all screenshot placeholders with actual captures.

7) Final Summary

Lab 6 deliverables were implemented: point cloud stitching, ICP benchmarking, and real-time RANSAC-based wall/floor detection with RViz2 visualization. The report structure and evidence sections are prepared; only placeholder screenshots and final numeric values need to be inserted from your latest execution.

- Lab report -
Lab 5:
Color Sensor

[S26] Sensors and Sensing

```
            **\- Innopolis University \-**
```

Anas Khmais Hamrouni
B23-RO-01

[Lab notebook]

[Lab Solution Git Repository]:

**experiments steps/instructions in README*

Topic :
Color sensor for basic color recognition using a photoresistor and RGB illumination .

Purpose:

To design, assemble, and calibrate a simple reflective color sensor that distinguishes red, green, and blue surfaces using controlled LED illumination and a photoresistor, and to test its behavior on mixed colors such as purple, yellow, and orange.

Description of the sensor application

The sensor is used to recognize the dominant color of an object placed in front of it by sequentially illuminating the object with red, green, and blue light and measuring the reflected intensity with a photoresistor. This approach can be applied in simple sorting systems

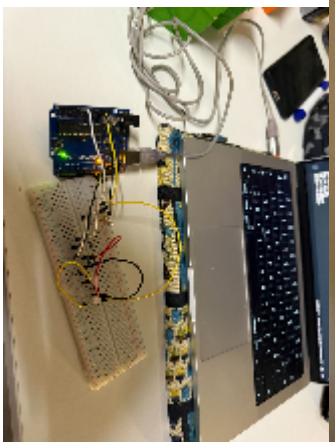
List of used equipment/stack:

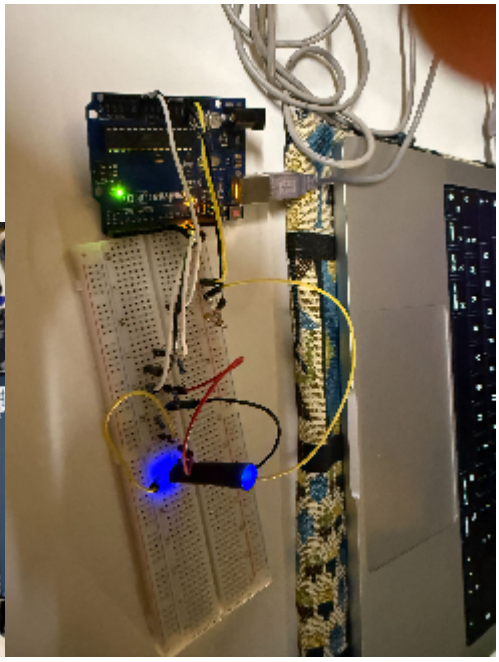
- **Hardware:** (documentation & datasheets in lab notebook)
  - Arduino Uno board
  - One RGB LED
  - One photoresistor
  - Resistors: three current-limiting resistors for the LED channels (220Ω) and one voltage-divider resistor for the photoresistor (10 kΩ).
  - Breadboard and jumper wires.
  - USB cable and computer with Arduino IDE

Description of experiments:

Experiment 1 — Sensor assembly

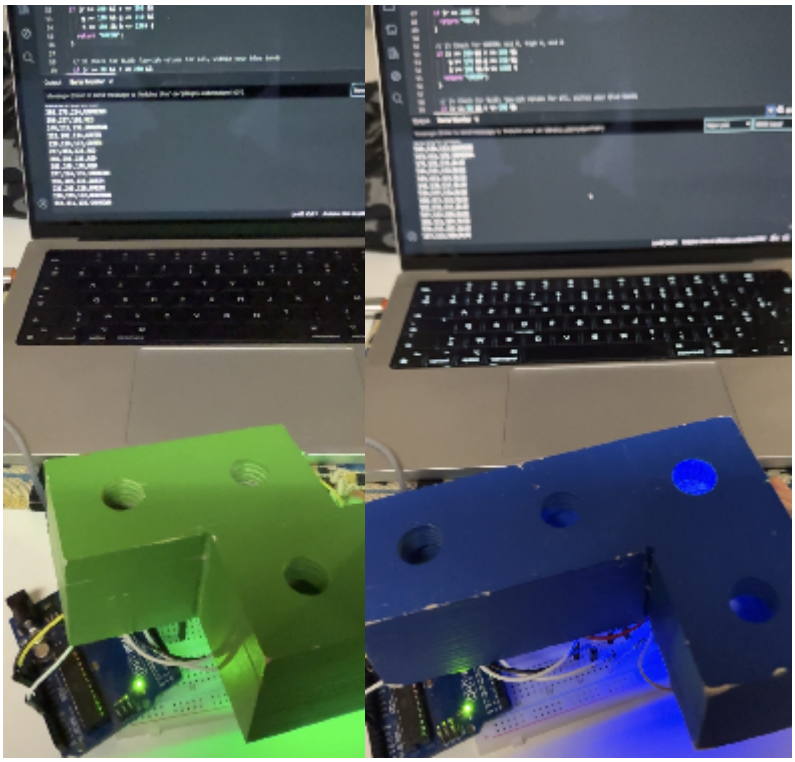
I connected the RGB LED channels to three Arduino digital PWM pins through current-limiting resistors and wired the photoresistor in a voltage divider to an analog input (A0) so that the analog reading decreases when less light is reflected.





Experiment 2 — Calibration procedure

For each base color (pure red, green, blue surfaces), I sequentially turned on the corresponding LED channel while keeping the other channels off and recorded multiple analog readings to determine typical ranges for “red object”, “green object”, and “blue object”.



Experiment 3 — Color recognition tests: (D_link)

I implemented an algorithm that, for every measurement cycle, turns on each LED channel one by one, reads the photoresistor value, and then compares the three values to decide which color channel produces the strongest reflected signal.

Experiment 4 — Mixed color measurements:

Using the same procedure, I measured the sensor responses on purple, yellow, and orange surfaces to see which base color the algorithm reports.

Description of the results of the experiments:

Experiment 2 — Calibration procedure

For calibration colors, the channel matching the object color produced the strongest reflection (distinctly different analog value) compared with the other two channels; this allowed me to choose thresholds

Experiment 4 — Mixed color measurements:

On a purple surface, the red and blue measurements were both relatively high reflection, while the green channel gave noticeably weaker reflection, so the algorithm tended to classify purple as red
On a yellow surface, red and green channels produced strong reflections and very similar raw values, while the blue channel produced much weaker reflection. With my implementation the algorithm chose red
On an orange surface, the red channel had the strongest reflection, green was moderate, and blue was clearly weakest. The algorithm classified orange as red

Analog Readings Records:

Surface Color	LED Red ON (average)	LED Green ON (average)	LED Blue ON (average)

Surface Color	LED Red ON (average)	LED Green ON (average)	LED Blue ON (average)
Red	317	181	163
Green	176	342	177
Blue	174	187	351
Purple	324	157	311
Yellow	332	333	148
Orange	348	230	155